Signed but Unprotected: A Formal and Empirical Study of MAVLink 2 Message-Signing Defaults and Their Bearing on the EU Cyber Resilience Act

Ian Brighouse
Brighouse Security Labs
Spain
research@brighousesec.com

***Abstract*—MAVLink is the de facto command-and-control (C2) protocol of the open-source drone ecosystem. Since 2016 MAVLink 2 has offered optional per-frame *message signing*, yet whether a deployed aircraft is actually protected depends on a tangle of implementation choices—default state, allow-lists, link exemptions, and a trust-on-first-use key-provisioning step—that, to our knowledge, no prior work has stated precisely or measured end to end. We make three contributions. First, we give a compact parameterised *acceptance semantics* for MAVLink 2 signing: a transition system whose parameters capture the points at which PX4 and ArduPilot differ, together with a bounded symbolic adversary and six security properties. Bounded exploration yields short, human-readable attack witnesses and, crucially, distinguishes an *idealised out-of-band* provisioning policy—which satisfies every property—from the policies real stacks implement, which do not. Second, we build three flight stacks from source (ArduPilot Copter 4.7.2, PX4 v1.17.0, PX4 v1.18.0-rc1) and audit them in software-in-the-loop with *mavaudit*, a passive, key-less auditor we release. We find that *all* default links are unsigned, answer unauthenticated reads, and expose MAVLink FTP; that the current *stable* PX4 release contains no signing implementation at all; and that with signing enabled ArduPilot's primary channel signs every outbound frame while still accepting unsigned commands—so a purely passive observer would wrongly judge it protected. This channel-0 exemption is a documented upstream behaviour (ArduPilot issue #34489, with a fix proposed but not yet merged [1], [2]); our contribution is to measure it independently, to show that the model built from the implementation reproduces it, and to make explicit that the exemption follows the channel index rather than the link type (measured by moving the transport of each channel independently), which motivates an active, read-only probe. Third, we map each observed property to the essential requirements of EU Regulation 2024/2847 (the Cyber Resilience Act) and the command-link requirement of Delegated Regulation 2019/945. We claim no primacy for that mapping: Annex I crosswalks for drone products are already published as compliance guidance, including ones that name unauthenticated MAVLink as the control-link weakness to remediate, and the four-column crosswalk form itself is established practice. What differs here is the content of the cells. Those crosswalks map a requirement to an intended control; each row of ours is anchored to a measurement taken from a running autopilot, with the record published alongside it, including the rows where the measurement was inconclusive and earned nothing. All code, models, and data are released for reproduction.**

***Index Terms*—MAVLink, unmanned aerial vehicles, message signing, protocol security, formal methods, Cyber Resilience Act, secure by default.**

I. Introduction

Open-source flight stacks—principally PX4 and ArduPilot—fly a large and growing share of commercial and dual-use unmanned aircraft. Both speak MAVLink, a message protocol designed for constrained radio links whose first version carried no authentication whatsoever [3]. Unauthenticated C2 has been shown repeatedly to enable mission disruption and takeover [3], [4], and the three protocol-level weaknesses are catalogued as CVE-2020-10281 through CVE-2020-10283. In 2026 a CISA advisory rated unauthenticated MAVLink in PX4 at CVSS 9.8 (CVE-2026-1579), and a cluster of memory-safety CVEs in MAVLink file transfer, the MAVLink shell, and log handlers showed that these sinks are reachable by *any* peer when signing is off.

MAVLink 2 added optional message signing that addresses integrity and origin authentication. Ten years later, three questions remain unanswered in the literature:

1) **Semantics.** *When exactly* does a receiver accept a frame? The specification leaves acceptance of unsigned traffic "to the implementation." What does that underspecification permit, and which choices are safe?
2) **Reality.** How are current releases configured out of the box, what does enabling signing actually change, and where are the exemptions?
3) **Consequence.** How do these facts map to the obligations EU drone manufacturers now carry under the Cyber Resilience Act (CRA) and the UAS product rules?

Contributions.

- A parameterised *acceptance semantics* for MAVLink 2 signing (Section III), with a bounded symbolic adversary and six properties. Exploration produces short attack witnesses and shows that an idealised out-of-band provisioning policy satisfies all six, while the policies PX4 and ArduPilot implement each violate a distinct subset.
- An empirical study of three stacks built from source (Section IV–V), using *mavaudit*, a released passive/active auditor. Key findings: insecure by default everywhere; no signing in stable PX4; and a sign-everything-yet-accept-unsigned behaviour on ArduPilot's primary channel that only an active probe detects. The behaviour itself is

tracked upstream [1], [2]; what we add is an independent measurement and a model that reproduces it from the implementation's own rules.
- A measurable crosswalk (Section VI) from protocol properties to CRA Annex I essential requirements and to the 2019/945 C2-link requirement, i.e. conformance evidence a manufacturer can actually produce.

All artefacts are available for reproduction.[1]

II. Background

a) Framing: MAVLink 1 frames begin with `0xFE` and a six-byte header; MAVLink 2 frames begin with `0xFD`, a ten-byte header, 24-bit message IDs, and *incompatibility* and *compatibility* flags. The one defined incompatibility flag, `0x01`, marks a signed frame. A per-message CRC detects mismatched message definitions but is not a security control.

b) Signing: A signed MAVLink 2 frame carries a 13-byte trailer: an 8-bit link ID, a 48-bit timestamp (units of $10\,\mu$s since 2015-01-01), and a 48-bit truncated SHA-256 over a 32-byte secret key, the header, the payload, the link ID, and the timestamp. Receivers must reject frames whose per-stream timestamp does not strictly increase, or that fall more than one minute behind local time.

c) What signing does not provide: (i) *Confidentiality*: payloads stay in plaintext. (ii) *Key distribution*: the 32-byte key is pushed with a SETUP_SIGNING message *in clear*, to be sent only over a secure link; there is no rotation, revocation, or per-device identity. (iii) *Per-sender identity*: any key holder can sign as any system/component. (iv) A *uniform* rule for unsigned traffic: acceptance is implementation-defined. These four facts are the reason a precise semantics is needed, and they drive the regulatory analysis in Section VI.

III. A Parameterised Acceptance Semantics

We model the receiver V as a transition system over states $\sigma = (K, armed, S, \mathrm{clk})$, where K is the current key (or $\perp$), $armed$ is the arming flag, S maps each stream $s = (sys, comp, link)$ to its last accepted timestamp, and clk is the local signing clock. A frame is $f = (\ell, src, \kappa, b, k, t, id)$ with link ℓ, claimed source src, kind $\kappa \in \{\mathrm{CMD}, \mathrm{READ}, \mathrm{HB}, \mathrm{RADIO}, \mathrm{SETUP}\}$, signed bit b, signing key k, timestamp t, and link id id. Signature verification is abstract: f *verifies* iff b and $k = K$ (no forgery; collisions ignored).

An implementation is a *policy* π—a tuple of the Boolean/parameter choices the specification leaves open (Table I): the set trusted of links that never require signatures; the set allow of kinds accepted unsigned while keyed; whether the receiver is *open when unkeyed*; whether it accepts the first key unsigned (*trust-on-first-use*, TOFU); whether an unsigned SETUP_SIGNING on a trusted link may replace the key; whether a signed SETUP_SIGNING may; and whether SETUP_SIGNING is refused while armed. The transition function $\mathrm{step}_\pi(\sigma, f) \to (accept, \sigma')$ is given in Algorithm 1; Fig. 1 shows the decision structure.

Algorithm 1 $\mathrm{step}_\pi(\sigma, f)$: acceptance decision

if $\kappa(f) = \mathrm{SETUP}$ **then** ▷ key management
  if π.setupLinks $\neq * \wedge\ \ell(f) \notin \pi$.setupLinks **then**
    return reject
  if π.refuseArmed $\wedge\ \sigma.armed$ **then**
    return reject
  if $\sigma.K = \perp$ **then**
    return π.tofu ? accept, $K \leftarrow k'$: reject ▷ TOFU window
  if f verifies $\wedge\ \mathrm{fresh}(\sigma, f) \wedge \pi$.signedRekey **then**
    return accept, $K \leftarrow k'$ ▷ signed rotation
  if $\neg b(f) \wedge \ell(f) \in \pi$.trusted $\wedge\ \pi$.unsignedRekey **then**
    return accept, $K \leftarrow k'$ ▷ console recovery
  return reject
if $\sigma.K = \perp$ **then**
  return π.openUnkeyed ? accept : reject
if $\ell(f) \in \pi$.trusted **then**
  return accept ▷ link exemption
if $b(f)$ **then**
  return $(k = K \wedge \mathrm{fresh})$? accept : reject
return $\kappa(f) \in \pi$.allow ? accept : reject

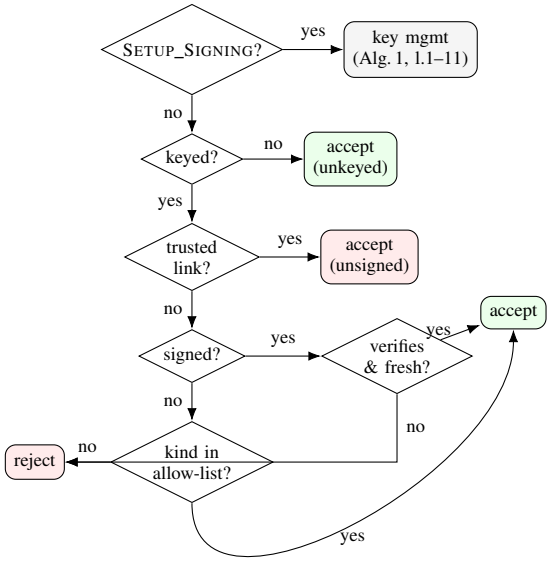


Fig. 1. Decision structure of step_π for an ordinary frame. Red sinks accept *unsigned* traffic; the link-exemption branch is the one an active probe is needed to observe (Section V, R4).

Three gaps in that table are worth naming, because they bound what the model run can be read as evidence *of*. First, no value in the armed column is measured on any stack: every probe in this work runs against a disarmed vehicle, so the armed-state row of each policy is a reading of the code and the documentation, and a stack that changes acceptance on arming would not have been caught here. Second, re-keying is measured on PX4 v1.18 only, where the record shows an

[1]Repository and data: https://github.com/brigghouse/brighouse-autonomy-lab.

TABLE I

POLICIES, WITH THE PROVENANCE OF *every cell* RATHER THAN OF EACH ROW. ^{m}: A RECORDED SITL MEASUREMENT BACKS THIS VALUE (SECTION V). ^{s}: EXTRACTED FROM THE STACK'S SOURCE WITH FILE AND LINE, NOT EXERCISED. UNMARKED: DOCUMENTATION- OR SPEC-DERIVED, AND EXERCISED ONLY AGAINST THE MODEL BY THE CONFORMANCE RUN. AN EARLIER VERSION OF THIS TABLE MARKED WHOLE POLICY ROWS *m*, WHICH CLAIMED MEASUREMENT FOR CELLS WE HAD NEVER MEASURED; THE RE-KEY AND ARMED-STATE COLUMNS ARE THE ONES THAT WERE OVERSTATED, AND THEY ARE UNMARKED HERE THROUGHOUT.

policy	trusted	allow	TOFU	u-rekey	s-rekey	armed
spec-strict	$\emptyset$	{radio}	yes	no	yes	no
ideal-oob	$\emptyset$	{radio}	no	no	yes	no
px4-1.18	$\emptyset^{m}$	{hb,radio}^{s}	yes^{m}	no^{m}	yes	yes
px4-1.17	all^{m}	—	no	—	—	—
ardupilot-4.7.2	{ch0}^{m}	{radio}^{s}	yes	yes	yes	no

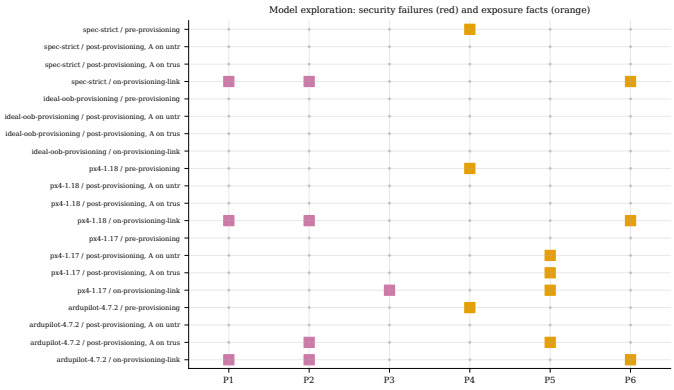


Fig. 2. Model exploration (depth 3). Red = security failure witnessed; orange = exposure fact present; dot = not witnessed. ideal-oob is clean throughout; the implemented policies are not.

unsigned re-key leaving the installed key unchanged and the original key still verifying every decoded frame; on ArduPilot we have no re-key record at all, signed or unsigned, so all three of its re-key cells are documentation-derived. Third, the allowlist cells come from source extraction rather than from the wire: we read which messages each stack exempts, and did not confirm by injection that an exempt message is in fact accepted unsigned. Each of the three is a candidate for the next campaign rather than a finding.

A. Adversary and properties

The adversary A sits on one link, can inject any unsigned frame and replay any frame it has observed on that link, and can sign only with a key it has learned. Because SETUP_SIGNING carries the key in clear, A observing provisioning on its link *learns the key*. A cannot forge. We check six properties by bounded breadth-first search over A's action sequences, recording the shortest witness: P1 command-integrity (no A-command accepted off a trusted link); P2 key-integrity (A cannot change K); P3 replay-resistance; P4 first-use window (A installs its own key before provisioning); P5 trusted-link exemption (A-command accepted on a trusted link); P6 key exposure (A on the provisioning link forges afterwards). P1–P3 are failures; P4–P6 are exposure facts.

B. Results of exploration

Fig. 2 summarises the outcome; representative witnesses follow. ideal-oob is the only policy clean on all six properties and in all scenarios—closing the TOFU window (no P4) removes the key-exposure chain as well. Every TOFU policy (spec-strict, px4-1.18, ardupilot-4.7.2) exhibits P4, and when A sits on the provisioning link it then achieves P1, P2, and P6: the key travels in clear, so a three-step trace—observe SETUP_SIGNING, then sign a command or a re-key—breaks integrity. px4-1.17, modelled as *everything-trusted* (no signing code; Section V), shows P5 in every post-provisioning scenario and P3 (no freshness check). ardupilot-4.7.2 additionally shows P2 with A on the trusted console link: an *unsigned* SETUP_SIGNING on `ch0` replaces the key.

The take-away for measurement is R4 below: a policy can sign every outbound frame yet still accept unsigned input (ArduPilot `ch0`). A passive observer cannot see this; the model tells us precisely which probe does.

IV. METHODOLOGY

mavaudit classifies each observed frame from its raw header bytes (MAVLink 1; unsigned MAVLink 2; signed MAVLink 2 via flag `0x01`) and identifies the autopilot from HEARTBEAT. It never needs the key. With an *active probe* it sends two unsigned, read-only requests to the first autopilot seen: a PARAM_REQUEST_READ for parameter index 0 (name-independent across releases—ArduPilot 4.7 renamed `SYSID_THISMAV` to `MAV_SYSID`) and a COMMAND_LONG requesting AUTOPILOT_VERSION (firmware and capability flags). Only replies for index 0 from the probed vehicle count; the COMMAND_ACK is recorded as corroboration. A scope guard restricts targets to loopback/private addresses and checks the sending peer before probing; an active probe on a serial port (which could egress over a radio) requires explicit authorisation. The tool is validated against a reference vehicle in unsigned, signed, signing-exempt, and MAVLink 1 modes.

We built three stacks from source on one headless Ubuntu 24.04 VM: ArduPilot Copter 4.7.2 (`fb77356789`), PX4 v1.17.0 (`d6f12ad1c4`, the latest stable), and PX4 v1.18.0-rc1 (`fca3df865a`). PX4 runs with its built-in SIH physics; no hardware, no RF. For each link we (i) audit the default configuration, (ii) provision the lab key as an operator would (an unsigned SETUP_SIGNING over the local link), (iii) audit again, and (iv) verify signatures with the key. For PX4 we also test that an unsigned re-key is rejected.

V. MEASUREMENT RESULTS

Fig. 3 and Table II give the measured data.

a) R1: insecure by default, everywhere: On all six default links: zero signed frames, the unauthenticated read answered, and MAVLink FTP advertised. A stack shipped unchanged provides a C2 link with neither integrity nor access control.

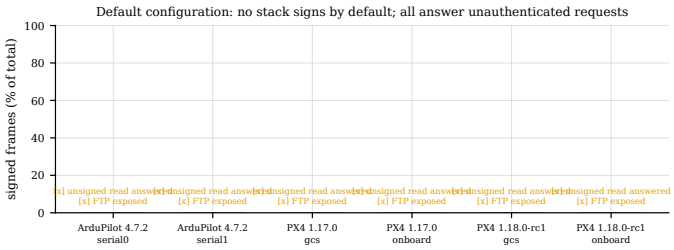


Fig. 3. Default configuration of all three stacks: zero signed frames; every link answers the unauthenticated read and exposes MAVLink FTP.

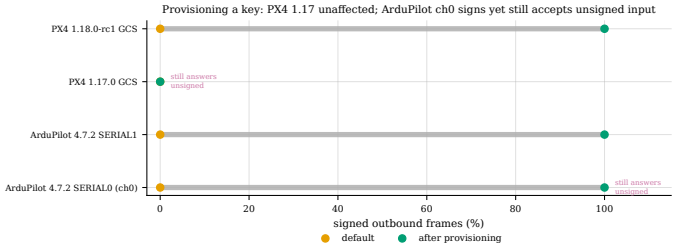


Fig. 4. Before vs after provisioning. PX4 1.17 is unchanged (no signing code). ArduPilot `ch0` and both PX4 1.18 links reach 100% signed, but `ch0` and PX4 differ on whether unsigned input is still accepted.

b) R2: stable PX4 cannot sign: In v1.17.0 the provisioning step had no effect—no key file, zero valid signatures, reads still answered. Source inspection agrees: the v1.17.0 MAVLink module contains no signing implementation and no `MAV_SIGN_CFG` parameter. The CISA remediation for CVE-2026-1579 ("enable signing") is thus unavailable on stable PX4; it first appears in the v1.18 line. This corrects a claim in our own earlier survey and, we believe, a plausible misreading of the v1.18 release notes.

c) R3: PX4 v1.18 signing matches its documentation: After unsigned first-use provisioning: a 40-byte key file appears; 100% of outbound frames are signed on both links and every decodable frame verifies; the unsigned read and version request go unanswered; and a later *unsigned* attempt to install a different key is rejected while the original key stays valid. The TOFU window before the first key remains (model P4).

d) R4: ArduPilot's channel-0 exemption hides from passive observers: With a key installed, SERIAL1 signs everything and ignores unsigned input. SERIAL0 also signs every outbound frame (Fig. 5), yet still answers the unsigned read and acknowledges the unsigned command request. This is by design: `accept_unsigned_callback` returns true on `MAVLINK_COMM_0`, "assumed to be secure channel . . . USB on ChibiOS boards." An auditor watching only traffic would conclude the link is protected. The model named this case (P5; Fig. 1, red link-exemption sink), which is why mavaudit's finding relies on an active read-only probe and states the discrepancy explicitly. A model-based conformance run [5] confirms it: over eight read-only probe cases the stack's behaviour matches the model exactly (8/8), including the keyed trusted-link case in which an unsigned read is answered.

Index, not transport. Because the exemption is justified by the assumption that channel 0 is USB, we checked whether it follows the channel index or the link type by moving the transport of each channel independently (Table III). With SERIAL0 carried over UDP instead of TCP the keyed channel still answers unsigned reads; with SERIAL1 carried over the same UDP transport it rejects them. The two runs differ only in which channel index the UDP link occupies, so the exemption is a property of the index. On any board whose SERIAL0 is a telemetry radio or a network port rather than USB, that port accepts unsigned commands while every other port is protected—the situation described in the upstream issue [1], now measured. A fourth configuration sets `SERIAL1_PROTOCOL`=1 (MAVLink 1): the documentation states such ports are "not protected," but in 4.7.2 a keyed MAVLink 1 port rejects an unsigned MAVLink 1 read (the frame cannot carry a signature and the channel is not channel 0); we report the measurement, which is stricter than the text. All 23 observations agree with the policy.

A second control plane the key does not cover: Signing protects MAVLink links, and a deployment that has provisioned a key may reasonably believe its command path is authenticated. PX4 ships a second one. The `uxrce_dds_client` module is compiled into 73 of the 286 board configurations in the v1.18.0-rc1 tree (12 disable it explicitly; its Kconfig default is *n*), but being compiled in is not the same as listening. The only startup script in the tree that starts it is the POSIX one, which runs it unconditionally with no parameter gate—unlike Zenoh two lines below, which is behind `ZENOH_ENABLE`. The stock NuttX startup script contains no reference to it at all, so a flight board can carry the module in flash with nothing listening on the port. The plane we measure below is therefore a property of POSIX targets—SITL, and Linux companion computers such as ModalAI's VoXL2, whose own start script also launches it—and does not

TABLE II
MEASURED OUTCOMES. "U-READ" = UNSIGNED READ ANSWERED; "KEYED" = SIGNATURES VERIFIED / FRAMES DECODED.

stack	link	default		provisioned	
		signed	u-read	signed	u-read
ArduPilot 4.7.2	SERIAL0 (ch0)	0%	yes	100%	**yes**
ArduPilot 4.7.2	SERIAL1	0%	yes	100%	no
PX4 1.17.0	GCS	0%	yes	0%	yes
PX4 1.17.0	Onboard	0%	yes	0%	yes
PX4 1.18.0-rc1	GCS	0%	yes	100%	no
PX4 1.18.0-rc1	Onboard	0%	yes	100%	no

TABLE III
CHANNEL-0 EXEMPTION VS. TRANSPORT, ARDUPILOT 4.7.2 SITL, KEY ACTIVE, UNSIGNED `PARAM_REQUEST_READ`. RECORDED IN `RESULTS/CH0-CONFIGS`.

Config	SERIAL0 (ch0)	SERIAL1 (ch1)	unsigned read answered
A	TCP	TCP	ch0 yes, ch1 no
B	**UDP**	TCP	ch0 **yes**, ch1 no
C	TCP	**UDP**	ch0 yes, ch1 **no**
D	TCP	TCP, MAVLink 1	ch0 yes, ch1 no (MAVLink 1 probe)

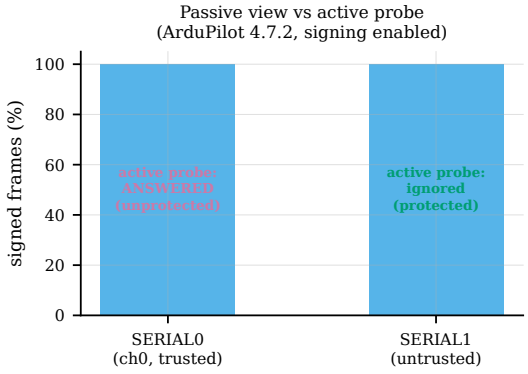


Fig. 5. ArduPilot 4.7.2 with signing enabled. Both links sign 100% of outbound frames (passive view identical); only the active probe reveals that `ch0` still accepts unsigned input.

transfer unexamined to a NuttX board. We state this precisely because an earlier version of this paper said "most firmware builds", which a case-sensitive search had appeared to support and the configuration symbol contradicts. It bridges uORB to a DDS network, and PX4's documentation states that the transport is unauthenticated and reaches uORB directly. We measured the consequence passively: binding the configured agent port and decoding what arrives, without answering. Both PX4 v1.17.0 and v1.18.0-rc1 emit repeated XRCE `GET_INFO` submessages addressed to `OBJECTID_AGENT` in a session with no client key—agent discovery offered to whichever process holds the port, with no credential, key or challenge preceding it, and no means for the client to authenticate the agent that answers. We did not answer, so no session was established and no topic was read or written; this is detection, not exploitation, and we claim nothing about what a full agent could then do beyond what PX4 documents. The same probe against ArduPilot 4.7.2 returns nothing and the build exposes no `DDS_ENABLE` parameter, so its DDS subsystem is not compiled in—a property of that build, since the source defaults the parameter to enabled where it is compiled.

The point for conformance is not that this is a defect; it is documented. The point is that *the key does not reach it*. A technical file that records "message signing enabled" as evidence for Annex I 2(d) is incomplete if a second, unauthenticated path to the same vehicle state ships in the same binary.

VI. From Protocol Properties to CRA Evidence

The CRA (Regulation (EU) 2024/2847) applies to products with digital elements that have a data connection; it excludes products *certified* under aviation Regulation 2018/1139, but open-category drones carrying a C-class mark under Delegated Regulation 2019/945 undergo conformity assessment, not certification, and are thus in scope, as is commercially distributed GCS software. A manufacturer integrating PX4 or ArduPilot carries the full Annex I obligations for the whole product. This is a technical interpretation, not legal advice. Table IV maps each measured observation to the Annex I essential requirement it bears on, to the 2019/945 C2-link requirement, and to the concrete evidence a manufacturer can produce with the released tooling.

TABLE IV
Crosswalk: protocol observation → CRA Annex I, Part I(2) → evidence. A *mavaudit* report (↓ severity) is machine-checkable conformance evidence.

observation	Annex I, Part I(2)	evidence
no signed frames	(b) secure by default; (d) access; (f) integrity	100% signed; provisioning procedure
answers unsigned read	(d); (j) attack surface	MAV-005 absent (active probe)
MAVLink 1 on radio port	(b); (f)	no v1 frames observed
no confidentiality	(e) confidentiality	encrypted datalink, risk-justified
FTP/shell exposed	(a) no known exploitable vuln.; (j)	services gated; SBOM + CVE watch
unsigned firmware	(f) integrity of programs	secure-boot configuration
no vuln. handling	Part II(5)–(6); Art. 14	CVD policy; ENISA runbook

The crosswalk is operational: each row is a finding mavaudit emits with a severity, so a CI gate (`mavaudit scan --fail-on high`) turns the CRA's "secure by default" requirement into a build-time check. The 2019/945 Part 3 requirement that a class-C2/C3 command link be "protected against unauthorised access" is outcome-only; our measurements show that MAVLink signing is a candidate mechanism but that, under R4, signing alone can fail a strict reading. The forthcoming EU Trusted Drone label and Drone Security Package [6] may specify the C2-link property more concretely; a measurable crosswalk such as ours is a candidate evidence basis.

VII. Discussion and Limitations

The study is simulation-only; vendor firmware and hardware radios may differ. Our legal reading tracks public texts as of November 2026. We report four defensive recommendations in priority order: (i) ship with signing enabled or an unavoidable first-run provisioning step, closing the TOFU window and preferring out-of-band keys (model: move from spec-strict toward ideal-oob); (ii) choose a stack version that can sign and treat any trusted/console link as unauthenticated; (iii) add link confidentiality where the risk assessment requires it; (iv) audit *actively*, since a link that signs everything may still accept unsigned input (R4).

VIII. Related Work

Foundational MAVLink security analyses [3], [4] and the MAVSec encryption proposal [7] established the threat but predate the signing deployment we measure. Fuzzing of flight stacks [8], [9] shows that authentication is necessary but not sufficient. Surveys [10], [11] catalogue threats without a

formal acceptance model or longitudinal measurement. Published regulatory crosswalks operate at the process level—CRA to ATT&CK and to GDPR [12], [13], UNECE R155 to ISO/SAE 21434 [14], DO-326A to avionics architecture [15]—and none maps essential requirements to *measurable protocol properties* of a drone C2 link or produces evidence by executing checks against the implementation. MAVLink 2 itself has been verified formally before: PAVE-MAVLink analyses it in ProVerif and validates the result empirically, over the air, on a physical PX4 aircraft [16], [17], proposing an added ChaCha20 layer as the remedy. We claim neither priority in verifying the protocol nor in testing against real hardware. What we have not found in prior work is a semantics *parameterised* by the points on which implementations differ—so that the stacks become instances to compare rather than separate case studies—or a conformance crosswalk that produces CRA evidence by executing checks against the implementation. We say this from published abstracts and metadata, the MILCOM proceedings being paywalled, and will revise it if the full texts show otherwise.

IX. Conclusion

MAVLink signing works when enabled: PX4 v1.18.0-rc1 and ArduPilot 4.7.2 SERIAL1 enforce it as documented. But it is off by default in every stack we tested, absent from the current stable PX4 release, and bypassed by design on ArduPilot's primary channel in a way no passive observer can see—a case our formal model predicts. For EU manufacturers the CRA turns this integrator responsibility into a legal obligation with live reporting duties. A reproducible audit that pairs passive observation with a read-only active probe is a cheap, machine-checkable starting point for conformance evidence—not a substitute for the assessment, but a row in it that someone else can re-run.

Availability and Ethics

All code, models, and data are released under permissive licences for reproduction. The work is defensive: no exploit code, attack automation, or RF tooling is included, and only self-hosted simulators were tested. The channel-0 signing exemption discussed here was already public upstream before this work (issue #34489 and the unmerged fix in PR #34780 [1], [2]); we therefore claim no new vulnerability, and no embargo applies. Our measurements are public and may be cited in that discussion.

References

[1] ArduPilot contributors, "MAVLink2 signing: unsigned packets accepted on channel 0 when channel 0 is a network link (Linux boards)," ArduPilot issue #34489, https://github.com/ArduPilot/ardupilot/issues/34489, 2026, accessed 3 Nov 2026.

[2] ——, "GCS_MAVLink: only exempt channel 0 from MAVLink2 signing when it is the USB port," ArduPilot pull request #34780 (opened 15 Oct 2026; unmerged as of 3 Nov 2026), https://github.com/ArduPilot/ardupilot/pull/34780, 2026, accessed 3 Nov 2026.

[3] J. A. Marty, "Vulnerability analysis of the MAVLink protocol for command and control of unmanned aircraft," MSc thesis, Air Force Institute of Technology, Mar. 2014, advisor: Barry E. Mullins. Theses and Dissertations 613. [Online]. Available: https://scholar.afit.edu/etd/613

[4] Y.-M. Kwon, J. Yu, B.-M. Cho, Y. Eun, and K.-J. Park, "Empirical analysis of MAVLink protocol vulnerability for attacking unmanned aerial vehicles," *IEEE Access*, vol. 6, pp. 43 203–43 212, 2018.

[5] I. Brighouse, "Acceptance semantics for MAVLink 2 signing: Bounded symbolic analysis and model-based conformance testing of flight stacks," 2026, companion paper; Brighouse Security Labs.

[6] European Commission, "Action plan on drone and counter-drone security," COM(2026) 81 final, Brussels, 11 Feb. 2026, CELEX 52026DC0081, 2026.

[7] A. Allouch, O. Cheikhrouhou, A. Koubaa, M. Khalgui, and T. Abbes, "MAVSec: Securing the MAVLink protocol for Ardupilot/PX4 unmanned aerial systems," in *2019 15th International Wireless Communications & Mobile Computing Conference (IWCMC)*, 2019, pp. 621–628, preprint: arXiv:1905.00265.

[8] T. Kim, C. H. Kim, J. Rhee, F. Fei, Z. Tu, G. Walkup, X. Zhang, X. Deng, and D. Xu, "RVFuzzer: Finding input validation bugs in robotic vehicles through control-guided testing," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 425–442. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/kim

[9] H. Kim, M. O. Ozmen, A. Bianchi, Z. B. Celik, and D. Xu, "PGFUZZ: Policy-guided fuzzing for robotic vehicles," in *Network and Distributed System Security Symposium (NDSS)*, 2021.

[10] M. A. Hamza, M. Mohsin, M. Khalil, and S. M. K. A. Kazmi, "MAVLink protocol: A survey of security threats and countermeasures," in *2024 4th International Conference on Digital Futures and Transformative Technologies (ICoDT2)*, 2024, pp. 1–8.

[11] B. Nassi, R. Bitton, R. Masuoka, A. Shabtai, and Y. Elovici, "SoK: Security and privacy in the age of commercial drones," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 1434–1451.

[12] J. Ruohonen, "SoK: The design paradigm of safe and secure defaults," *Journal of Information Security and Applications*, 2025, arXiv:2412.17329.

[13] J. Ruohonen, K. Hjerppe, and E.-Y. Kang, "A mapping analysis of requirements between the CRA and the GDPR," in *Proc. IEEE 33rd International Requirements Engineering Conference Workshops (REW)*, 2025, arXiv:2503.01816.

[14] G. Costantino, M. De Vincenzi, and I. Matteucci, "A comparative analysis of UNECE WP.29 R155 and ISO/SAE 21434," in *Proc. IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, 2022.

[15] F. Siddiqui, A. Ahlbrecht, R. Khan, S. Tasdemir, H. Hui, B. Sonigara, S. Sezer, K. McLaughlin, and W. Dürr, "Cybersecurity engineering: Bridging the security gaps in avionics architectures and DO-326A/ED-202A," in *Proc. IEEE/AIAA 42nd Digital Avionics Systems Conference (DASC)*, 2023, [VERIFY full author list].

[16] T. Wray and Y. Wang, "PAVE-MAVLink: Formal verification of MAVLink 2 for secure UAV communications," in *Proc. IEEE Military Communications Conference (MILCOM)*, Los Angeles, CA, USA, 2025.

[17] ——, "PAVE++ demo: Cross-layer formal verification and OTA validation for UAV communications," in *Proc. IEEE Military Communications Conference (MILCOM)*, Los Angeles, CA, USA, 2025, pp. 915–916, demonstration: ProVerif analysis of MAVLink 2, simulated attacks on the Damn Vulnerable Drone platform, and over-the-air validation on a physical PX4 aircraft, with ChaCha20 encryption as the mitigation.